Supplementary Materials

Understanding How Humans Inject Knowledge into Machine Learning Workflows through Visual Analytics

Y. Xing[1], P. Beaucamp[1], J. Chakraborty[1], A. Farea[2], Y. Jin[1], S. Khan[3], G. Andrieko[4 5], N. Andrienko[4 5], & M. Chen[1]

[1]University of Oxford, UK
[2]Istanbul Technical University, Turkiye
[3]Science and Technology Facilities Council (STFC), UK
[4]Fraunhofer Institute IAIS, Germany
[5]City St George's University of London, UK

Supplementary Materials A: Complete List of Code Categories and Definitions

1 Machine Learning (ML) Related Codes:

1.1 **ML_TECH**: ML Techniques

Categorizations of commonly used ML techniques and methods, ranked in alphabetical order.

1. **Association Rules**: A rule-based method for discovering relationships or co-occurrence patterns between variables.
2. **Autoencoder (AE) / Variational Autoencoder (VAE)**
3. **Bagging**: An ensemble method that trains multiple models on bootstrap samples of the dataset and aggregates their predictions to improve robustness.
4. **Bayesian Networks / Belief Networks**: Probabilistic graphical models representing variables and their conditional dependencies using directed acyclic graphs.
5. **Boosting (Gradient Boosting, XGBoost, AdaBoost, LightGBM)**: Ensemble methods that sequentially train models, where each model focuses on correcting errors made by previous models.
6. **Convolutional Neural Networks (CNNs)**: Deep neural networks using convolutional layers to capture spatial patterns in grid-like data such as images.
7. **Density-based clustering**: A density-based clustering algorithm that groups points based on local density and identifies noise or outliers.
8. **Decision Rules**: Interpretable models that classify instances using a set of logical if–then rules derived from data.
9. **Decision Trees**: Tree-structured models that recursively split data based on feature conditions to make predictions.
10. **Deep Reinforcement Learning**: Reinforcement learning methods that use deep neural networks to approximate policies or value functions.
11. **Diffusion Models**: Generative models that learn to generate data by reversing a gradual noise diffusion process.
12. **Flow-based Models**
13. **Gated Recurrent Unit (GRU)**
14. **Gaussian Mixture Models (GMM):** Probabilistic models representing data as a mixture of multiple Gaussian distributions.

15. **Gaussian Processes (GP) Regression/Classification**: Non-parametric Bayesian models that define distributions over functions for regression or classification tasks.
16. **Generative Adversarial Networks (GANs)**: Generative models consisting of a generator and discriminator trained adversarially to produce realistic data.
17. **Genetic Algorithms (GA)**
18. **Genetic Programming (GP)**
19. **Graph Neural Networks (GNNs)**: Neural networks designed to learn representations from graph-structured data using message passing between nodes.
20. **Hidden Markov Models (HMM)**: Probabilistic models for sequential data where hidden states generate observable outputs through Markov transitions.
21. **Hierarchical Clustering**: A clustering method that builds nested clusters through iterative merging or splitting.
22. **k-means Clustering**: A centroid-based clustering algorithm that partitions data into k clusters by minimizing within-cluster variance.
23. **k-Nearest Neighbours (k-NN)**: A non-parametric method that predicts labels based on the majority class of the k nearest data points.
24. **Kernel Fisher Discriminant**
25. **Kernel PCA**
26. **Kernel Ridge Regression**: Regression combining ridge regularization with kernel methods for nonlinear prediction.
27. **Kolmogorov-Arnold Network (KAN)**
28. **Large Language Models / Transformers**: Neural architectures using attention mechanisms to model long-range dependencies in sequential data such as text.
29. **Linear / Non-Linear SVM**: Support Vector Machines that find optimal separating hyperplanes, with kernels enabling nonlinear decision boundaries.
30. **Linear Discriminant Analysis (LDA)**: A statistical method for classification and dimensionality reduction based on maximizing class separability.
31. **Linear Regression**: A statistical model that predicts a continuous output as a linear combination of input variables.
32. **Logic Expressions / Expert Systems**
33. **Logistic Regression**: A classification model that estimates class probabilities using a logistic function.
34. **Long Short-Term Memory (LSTMs)**: Recurrent neural networks designed to capture long-term dependencies using memory cells and gating mechanisms.
35. **Markov Random Fields (MRF)**
36. **Multi-Layer Perceptron (MLPs)**: Fully connected feedforward neural networks consisting of multiple layers of nonlinear transformations.
37. **Naive Bayes**: Probabilistic classifiers based on Bayes' theorem with an assumption of conditional independence between features.
38. **Neural Architecture Search (NAS):**
39. **Neuro-symbolic Models**
40. **Ontology Learning**
41. **Physics-informed Machine Learning (PIML) / Physics-informed neural networks (PINN)**: Models that incorporate physical laws or differential equations as constraints during training.
42. **Principal Component Analysis (PCA)**: A linear dimensionality reduction method that projects data onto directions of maximal variance.
43. **Quadratic Discriminant Analysis (QDA)**: A probabilistic classifier allowing quadratic decision boundaries by modeling class-specific covariance matrices.
44. **Recurrent Neural Networks (RNNs)**: Neural networks designed to process sequential data by maintaining hidden states across time steps.
45. **Restricted Boltzmann Machines (RBM)**

46. **Ridge, Lasso, and Elastic Net Regression**: Regularized regression methods that control model complexity through L2, L1, or combined penalties.
47. **Self-Organizing Maps (SOM)**
48. **Stacking / Voting Classifiers**
49. **Support Vector Regression**: A regression version of Support Vector Machines (SVM) that predicts continuous values using margin-based optimization.
50. **Vision Transformers (ViT)**: Transformer-based architectures that process images as sequences of patches using attention mechanisms.
51. Others

1.2 **ML_TM**: ML Training Methods

Common approaches for training the model.

1. **Supervised Learning**: Models are trained using labelled data, where each training instance is paired with a ground-truth label that guides the learning process.
2. **Unsupervised Learning**: Models are trained using unlabelled data to discover underlying patterns, structures, or distributions within the data.
3. **Self-Supervised Learning** [2]: Such models generate their own labels from the input data (e.g., predicting the next word in a sentence)
4. **Semi-Supervised Learning** [3]: Uses a small amount of labelled data and a large amount of unlabelled data.
5. **Reinforcement Learning**: Models learn by interacting with an environment and receiving feedback in the form of rewards or penalties, aiming to learn a policy that maximizes cumulative reward over time.
6. **Evolutionary** [9]: Training via heuristic search, mutation, and selection (e.g., the training method used for genetic programming).
7. **Transfer Learning**: A pre-trained model is adapted or fine-tuned for a new, related problem. Such learning can be both supervised (e.g., a pre-trained model is fine-tuned using labelled data) and unsupervised.
8. Others

1.3 **ML_ARCH**: ML Architecture

Architectures or families of architectures based on the structure of ML models.

1. **Mathematical Expressions** [9, 12]: Consists of models fundamentally defined by equations and continuous transformations, mapping inputs to outputs via direct algebraic formulas [12]. Common examples include Linear Regression, Logistic Regression, and Principal Component Analysis. Furthermore, this structural family encompasses the outputs of Symbolic Regression, where evolutionary training methods like Genetic Programming (GP) are used to iteratively search the space of mathematical expressions to discover equations represented as abstract expression trees [13].
2. **Rule-Based Models** [4]: Models are structured around discrete logic expressions or a strict set of deterministic rules. They rely on Boolean logic (IF/THEN statements) rather than continuous weights. Examples: Expert Systems, Association Rule Learning (e.g., Apriori).
3. **Tree-Based Models** [4]: Models structured as a branching, hierarchical directed graph where decision nodes split data based on feature values. Non-linear splits that segment the feature space into distinct regions. Examples: Decision Trees; Genetic Algorithms (GA) are frequently used to optimize the weights of Feedforward or Recurrent Computational Models (instead of using Backpropagation).

4. **Instance-based Models** [4]: Models whose primary architecture is the memorized training dataset itself. They do not abstract the data into weights or rules during training; instead, they compute similarity metrics directly against the stored data at runtime. Examples: k-Nearest Neighbours (k-NN).
5. **Kernel Methods**: Models structured around kernel functions that implicitly map input data into higher-dimensional feature spaces to make it linearly separable. They rely on computing the inner products between the images of all pairs of data in the feature space (the "kernel trick"). Examples: Support Vector Machines (SVMs), Kernel Ridge Regression, and Gaussian Mixture Models (GMMs).
6. **Feedforward Computational Models** [4]: Models structured as a Directed Acyclic Graph (DAG) where information flows in one direction without internal feedback loops. This may include: a) Standard Artificial Neural Networks / Feedforward Networks, MLPs, CNNs, U-Nets; b) some Transformers; c) Graph Neural Networks; d) Some Generative models implemented as feedforward transformations, Normalizing Flow, and Denoising Diffusion Probabilistic Model.
7. **Recurrent Computational Models** [11]: Models whose structure includes feedback loops. This may include: a) Recurrent sequence models such as Recurrent Neural Network, Long Short-Term Memory; b) Energy-based / cyclic neural networks, such as Restricted Boltzmann Machine; c) Undirected probabilistic graphical models, and Markov Random Field.
8. **Parameterized Arbitrary Programs**: Models whose structure is defined as a computational program, which is controlled by a set of learnable parameters. The program can be an arbitrary program, e.g., written in Java or C++. Examples of this class include genetic algorithm (optimizing a parameterized program) and AutoML (optimizing an ML training process).
9. **Ensemble Models**: Meta-structures that combine multiple independent, structurally distinct base models to produce a single aggregated prediction. The overarching structure is a voting, averaging, or sequential correction mechanism. Examples: Bagging (Random Forests), Boosting (which can be applied to trees, expressions, or DAGs), and Stacking.
10. **Hybrid Models** [10]: Models that fundamentally fuse two or more different architectural structures directly into a single, end-to-end pipeline. Examples: Neuro-symbolic AI (mixing DAGs with Rule-based models) or mixing continuous regression layers within a decision tree.
11. Others

1.4 **ML_GRP**: ML Grouping

The commonly used ML groupings.

1. **LG1-Linear:** linear / GLM [X,W]
2. **LG2-LogSym:** logic / symbolic / rule-based [X, Y, ZZ]
3. **LG3-StatsGraphica:** statistics / Bayesian / graphical [X, Y]
4. **LG4-Instance:** instance-based (including nearest neighbour) [Y]
5. **LG5-Kernel:** SVM and kernels [X, Y]
6. **LG6-Neural**: neural [X, ZZ]
7. **LG7-Seq**: RNN / sequence models (including HMM) [Z, W]
8. **LG8-DT**: DT [X, Y]
9. **LG9-Autoencoder**: autoencoder [Z]
10. **LG10-Perception**: perception-based [Y]
11. **LG11-Ensemble**: ensemble (RF here) [X]
12. **LG12-Evolution**: evolutionary [K, DP]
13. **LG13-Others**: others

[X]Fernández-Delgado et al. (JMLR 2014) families: they state they evaluate many classifiers “from 17 families” and list examples including discriminant analysis/GLM, Bayesian, neural networks, SVM, decision trees, rule-based classifiers, and ensemble families such as boosting, bagging, stacking, and random forests. "Do we Need Hundreds of Classifiers to Solve Real World Classification Problems?"

[Y]Kotsiantis (2007) technique families: the review organizes classification techniques into families such as Logical/Symbolic, Perceptron-based, Statistical (incl. Bayesian networks), Instance-based, and Support Vector Machines. "Supervised Machine Learning: A Review of Classification Techniques"
[Z]Goodfellow et al. (2016) architecture families: organizes neural/deep methods in major parts/chapters. "Deep Learning. MIT Press."
[W] Bishop (PRML, 2006). "Pattern Recognition And Machine Learning"
[ZZ] Wan (2024) families."TOWARDS COGNITIVE AI SYSTEMS: A SURVEY AND PROSPECTIVE ON NEURO-SYMBOLIC AI": describes neuro-symbolic AI as fusing neural and symbolic (and often probabilistic) approaches to improve interpretability/robustness.
[K] Eiben & Smith (2015) (for Evolutionary Algorithm / Genetic Algorithms). "Eiben, Agoston E., and James E. Smith. Introduction to evolutionary computing. Springer, 2015.
[DP] Blondel et al. (2024) (for differentiable programming: optimizing learnable parameters inside computational programs via automatic differentiation).

1.5 **ML_TASK**: Machine Learning Task

High-level categorization of the model objectives/problems the user is trying to solve.

1. **Classification**: Predicting discrete class labels for input data instances.
2. **Recognition / Detection**: Identifying and locating specific objects, events, or patterns within data (e.g., object detection in images).
3. **Regression**: Predicting continuous numerical values from input variables.
4. **Clustering**: Grouping data points into clusters based on similarity.
5. **Association**: Discovering relationships or co-occurrence patterns between variables in datasets.
6. **Feature Selection**: Identifying the most relevant subset of features for improving model performance or interpretability.
7. **Dimensionality Reduction**: Reducing the number of variables while preserving important data structure or information.
8. **Anomaly / Outlier / Adversarial Detectio**n: Identifying unusual, rare, or suspicious instances that deviate from normal patterns.
9. **Segmentation**: Partitioning data into meaningful regions or segments (e.g., pixel-level classification in images).
10. **Uncertainty Quantification**: Estimating the confidence or uncertainty associated with model predictions.
11. **Time Series Forecasting / Trend Analysis**: Predicting future values or identifying trends in sequential temporal data.
12. **Generation (Generative AI)**: Question answering, and text/audio/video/image generation.
13. **Named Entity Recognition**:
14. **Natural Language Processing**: Processing and analyzing human language data for tasks such as translation, summarization, or sentiment analysis.
15. Others

1.6 **ML_WF**: Machine Learning Workflow

1. **Development**: Workflow stages focused on training, testing, and improving ML models, typically involving ML researchers or developers.

2. **Deployment**: Workflow stages focused on using and evaluating deployed ML models in real-world applications, typically involving end-users or third parties.
3. **Both**

1.7 **ML_USR**: Machine Learning User Type

1. **Developer**: ML researchers or engineers who design, train, test, and improve machine learning models or systems.
2. **End User**: Individuals who use the deployed ML system to perform specific tasks without directly modifying the underlying model.
3. **Third Party**: Independent analysts or stakeholders who evaluate, monitor, or analyze ML systems without being involved in their development or primary use.

1.8 **ML_IN**: Machine Learning Input Data

1. **Text**: Natural language data represented as sequences of words, tokens, or characters.
2. **Tabular**: Structured data organized in rows and columns.
3. **Tree, Set, Graph/Network**: Structured relational data representing hierarchical structures (trees), unordered collections (sets), or interconnected entities (graphs/networks).
4. **Geospatial**: Data associated with geographic locations or spatial coordinates.
5. **Time-series**: Sequential data indexed by time, representing measurements or observations over time.
6. **Image:** Data represented as a two-dimensional grid of pixels encoding color or intensity information.
7. **Audio**: Time-varying sound signals that are typically represented as waveforms or spectrograms.
8. **Video**: Sequential visual frames forming spatiotemporal visual data.
9. **Tensor**: Multi-dimensional numerical arrays (≥3 dimensions) that do not fall into the previous categories, such as tabular data, image, audio, or video.
10. **Multi-modality**: Data composed of multiple modalities combined together (e.g., image and text, audio and video).
11. Others

1.9 **ML_OUT**: Machine Learning Output Result

1. **Label**: Discrete classes/label, either numerical or textual.
2. **SingleValue**: Single number/token.
3. **SequenceValues**: Sequence numbers, tokens, words, or characters, including time-series data.
4. **Tabular**: Structured data organized in rows and columns.
5. **ComplexStructured**: Complex objects including trees, sets, graphs, bounding boxes, and segments.
6. **Embedding**: High-dimensional vectors (e.g., output of any encoder such as autoencoder, word2vec, and Transformer).
7. **Ranking**: Ordered lists (recommendation systems).
8. **Geospatial**: Data associated with geographic locations or spatial coordinates.
9. **Image:** Data represented as a two-dimensional grid of pixels encoding color or intensity information.
10. **Audio**: Time-varying sound signals that are typically represented as waveforms or spectrograms.
11. **Video**: Sequential visual frames forming spatiotemporal visual data.
12. **Tensor**: Multi-dimensional numerical arrays (≥3 dimensions) that do not fall into the previous categories, such as tabular data, image, audio, or video.
13. Others

2 Visualization Related Codes

2.1 **VIS_MLS**: ML Stages Supported by Visualization

Codes in this section categorize which stage of the machine learning workflow is supported by visualization, as reported in the paper. The focus is on where visualization is used to assist human understanding and decision-making during the ML process. Supported by [14].
*Prefix meaning: **Dv** = Development, **Dp** = Deployment.*

1. **Dv_PrepareData**: Visualizations used during the data preparation stage, supporting activities such as data inspection, cleaning, transformation, feature extraction, synthetic data generation, or partitioning of training/validation/test sets.
2. **Dv_PrepareLearning**: Visualizations used to support the configuration of the learning process, including model selection, hyperparameter configuration, feature engineering decisions, and preparation of learning pipelines prior to model training.
3. **Dv_TrainModel**: Visualizations used during the model training process to monitor or analyze training dynamics, such as loss curves, gradient behavior, parameter updates, convergence behavior, or internal model states.
4. **Dv_EvaluateModel:** Visualizations used for evaluating trained models, supporting performance analysis, error inspection, model comparison, validation diagnostics, or interpretation of evaluation metrics.
5. **Dp_UseDeployedModel**: Visualizations used when using an already deployed model, supporting decision-making, prediction interpretation, or exploration of model outputs in real-world use contexts.
6. **Dp_ThirdPartyMonitor**: Visualizations used by third-party stakeholders to monitor deployed ML systems, including monitoring system performance, model drift, operational metrics, reliability, or overall system health.
7. Others

2.2 **VIS_OBJ**: Visualized Object

Codes in this section categorize which ML–related objects or data entities are visualized in the system described in the paper.

1. **InputData**: The raw input data provided to the ML model, including original datasets, samples, images, text, signals, or tabular data before feature engineering.
2. **GroundTruthLabel**: Ground-truth labels or annotations associated with the input data, typically used for supervised learning tasks.
3. **EngineeredFeature**: Engineered or derived features generated from raw data through preprocessing, feature extraction, or feature engineering methods.
4. **LatentFeature:** Latent representations produced by the model (e.g., embeddings, hidden vectors, feature maps), typically representing compressed or abstracted representations of input data.
5. **InternalStates**: Internal model states or parameters during computation, such as neuron activations, gradients, attention weights, or parameter values.
6. **PerformanceData**: Model performance metrics or evaluation statistics, such as accuracy, precision, recall, F1 score, loss values, confusion matrices, or other performance indicators.
7. **PredictedLabel**: Predicted labels generated by the ML model for classification or labeling tasks.
8. **OutputResult**: Model outputs beyond predicted labels, including regression predictions, probability distributions, ranking results, generated content, or decision scores.
9. **TrainingProgression**: Temporal records of the training process, including epoch progression, loss curves, accuracy changes, or other indicators of training dynamics.

10. **ModelArchitecture**: The structural design of the ML model, including network layers, computational modules, connections, or pipeline components.
11. Others

2.3 **VIS_TYP**: Plot Types Used in Visualizations

Codes in this section categorize the structural type of visualization used to represent data or ML-related information. Partially supported by [15].

1. **CommonStatistics**: Basic statistical charts used to display numerical relationships or distributions, such as bar charts, line charts, scatter plots, area charts, histograms, or box plots, etc...
2. **ComplexTabular / Multivariate**: Visualizations designed to represent tabular data and multiple variables simultaneously, such as parallel coordinates, scatterplot matrices, or heatmaps.
3. **ComplexEntity-Relation (Graph)**: Visualizations representing relationships between entities, such as node-link diagrams, chord diagrams, adjacency matrix, or other types of visualizations for representing entity relations.
4. **ComplexHierarchical (Tree):** Visualizations designed to represent hierarchical or tree-structured data, such as treemaps, sunburst charts, and dendrograms.
5. **ComplexProcess / Flow:** Visualizations representing processes, pipelines, or flows of information, such as flow diagrams, Sankey diagrams, Alluvial diagrams, or pipeline visualizations.
6. **ComplexGeospatial**: Visualizations representing geographically referenced data using maps or spatial layouts, such as choropleth maps or symbol maps.
7. **ComplexTemporal**: Visualizations representing data evolving over time, such as timelines, time-series plots, or calendar visualizations.
8. Others

2.4 **VIS_CH**: Visual Channels Used in Visualizations

Codes in this section categorize the visual channels used to represent information in the visualization. Supported by [16].

1. **Geometric**: Encoding information through geometric properties such as position, size (length, width, depth), orientation, shape, curvature, or smoothness.
2. **Optical**: Encoding information through optical properties such as color (hue, saturation, brightness), opacity/transparency, texture, line style, shading and lighting effects, depth cues, or motion.
3. **Topological and Relational**: Encoding information through spatial or relational structure, such as connectivity, adjacency, containment, intersection/overlap, depth ordering/partial occlusion, or closure.
4. **Semantic:** Encoding information through symbolic or semantic elements such as numbers, texts, symbols/ideograms, icons/glyphs/pictograms, isotype, or other symbolic representations.
5. Others

2.5 **VIS_PTN**: Patterns Discovered from Visualizations

Codes in this section categorize the types of patterns, insights, or knowledge that users can discover from the visualization, reflecting the analytical questions supported by the visualization.

1. **Outlier**: Identification of anomalous data points, unusual model behavior, or unexpected patterns in the data or model outputs.

2. **Distribution:** Understanding the distributional characteristics of data, features, model outputs, or model errors, such as skewness, variance, or density patterns.
3. **Cluster**: Identification of groups or clusters of similar data points, model behaviors, or feature representations.
4. **EmbeddingSpace**: Understanding the structure or organization of learned embedding spaces or latent representations.
5. **Error/Failure**: Identification and analysis of model errors, failure cases, or misclassifications.
6. **Uncertainty**: Understanding uncertainty in model predictions, outputs, or model confidence.
7. **Drift/Shift**: Identification of distribution shifts or data drift between training and deployment environments.
8. **Influence**: Understanding how particular data points, features, or components influence model predictions or behavior.
9. **Bias/Fairness/Confusion**: Analysis of bias, fairness issues, or confusion patterns in model predictions across groups or classes.
10. **TradeOff:** Exploration of trade-offs between competing objectives such as accuracy, robustness, fairness, efficiency, or interpretability.
11. **Model/ArchitectureDifference**: Comparative analysis between multiple models, architectures, or configurations.
12. **ModelBehaviour**: Understanding the general behavior or response patterns of the model under different inputs, conditions, or parameter settings.
13. **Relationship**: Understanding of how model results or errors relate to data dimensions (time, space, population structure, etc.); e.g., urban vs. rural areas.
14. Others

***2.6 VIS_NoC:** An extra column to capture the number of visualization components or views that are used in the VA system.

3 Interaction Related Codes

3.1 **INT_MLS**: Interaction by ML Stage

In which part of the ML workflow do humans use interactions.
*Prefix meaning: **Dv** = Development, **Dp** = Deployment.*

1. **Dv_PrepareData**: Interactions used for data cleaning, labeling, sampling, or generation.
2. **Dv_PrepareLearning**: Interactions used for comparing or selecting model components, architecture, or pretrained models before training.
3. **Dv_TrainModel**: Interactions used during training progression.
4. **Dv_EvaluateModel**: Interactions used for evaluating model results.
5. **Dp_UseDeployedModel**: Interactions used when using an already deployed model, supporting decision-making, prediction interpretation, or exploration of model outputs in real-world use contexts.
6. **Dp_ThirdPartyMonitor**: Interactions used by third-party stakeholders to monitor deployed ML systems, including monitoring system performance, model drift, operational metrics, reliability, or overall system health.
7. Others

3.2 **INT_TYP_NAV**: Interaction Types for Navigation / Visual Exploration *[17, 18, 19]*

Categorizes interactions that allow users to navigate, filter, and visually explore data, models, or results within interactive visualizations.

1. **Filter/Slice**: Filtering or slicing subsets of data, cohorts, or segments.
2. **Select/Focus**: Selecting items or regions (click, lasso, selection tools).
3. **Brush/Link**: Brushing and linked highlighting across multiple coordinated views.
4. **Zoom/Pan**: Zooming or panning within visualizations.
5. **Sort/Reorder**: Sorting or reordering axes, items, features, or models.
6. **DetailsOnDemand**: Tooltips or drill-down panels showing detailed information.
7. **Search/Query**: Search or query interface for items, features, or samples.
8. **Others**

3.3 **INT_TYP_MLDv**: Interaction Types for ML Development Workflow Steering

Categorizes interactions through which users steer, configure, and manage the machine learning development workflow.

1. **ParameterAdjust**: Adjust model parameters or hyperparameters.
2. **TriggerIteration**: Trigger retraining, recomputation, or optimization of the ML model.
3. **RunManage**: Save, restore, or manage experiment runs or model versions.
4. **ModelSelect:** :Select one or more trained models, checkpoints, branches, baselines, or experiment runs for focused inspection or side-by-side evaluation.
5. **MetricSelect:** Select one or more evaluation metrics or scoring criteria to guide inspection, ranking, or comparison of model behaviour or performance.
6. **ArchitectureSelect**: Select architectures or architectural components for inspection/comparison (e.g., ResNet vs ViT; encoder vs decoder blocks; layers, heads, modules, skip connections; structural variants).
7. **ConstraintRuleEdit**: Define or edit rules, constraints, or objectives.
8. **DataAdjust:** Define or adjust the structure of data utilised in modeling (e.g., cohorts, segments, partitions, units of analysis, training samples, or datasets)
9. **FeatureSelectTransform**: Select or transform features.
10. **RelationshipAdjust**: Adjust similarity/distance measures or grouping rules.
11. Others

3.4 **INT_TYP_MLDp**: Interaction Types for ML Deployment Workflow Steering

Categorizes interactions used to inspect, control, and adapt deployed machine learning systems during operational use.

1. **PredictionInspect**: Inspect predictions produced by the deployed model.
2. **ExplainInspect**: Inspect explanations for deployed model predictions.
3. **PerformanceMonitor**: Monitor deployed model performance over time.
4. **DataDrift**: Monitor incoming data distributions or detect concept/data drift relative to the training data.
5. **ErrorAnalysis**: Analyse errors or misclassifications produced by the deployed model to identify systematic failures.
6. **DecisionSupport**: Human interaction with AI recommendations in an operational decision workflow.
7. **ModelAdjust**: Configuring a model before the model performs the next tasks, e.g., setting parameters of models (e.g., threshold), changing the weights of sub-models in an ensemble model.
8. **DecisionAdjust**: Adjusting operational decisions or inputs applied to model outputs, e.g., selecting recommendations, modifying prompts, or controlling preprocessing parameters.
9. **FeedbackCollect**: Provide feedback on predictions.

10. **WhatIfDp**: What-if simulation or scenario probing.
11. Others

3.5 **INT_TYP_AnF**: Interaction Types for Annotation / Human Feedback

Categorizes interactions through which humans provide annotations, labels, or corrective feedback to support model learning or evaluation.

1. **LabelAnnotation**: Manual labeling or annotation
2. **ActiveLearning**: Query selection and label; a learning algorithm interactively queries a human user to label new data points with desired outputs.
3. **ErrorConfirm**: User confirms or corrects model errors.
4. **Others**

3.6 **INT_MOD**: Interaction Modalities

Categorizes the interface modalities through which users interact with ML systems or visual analytics tools.

1. **GUIManipulate**: Manipulation of visual objects and UI components mapped to ML changes.
2. **ProgrammingUI**: Manipulate through programming interface, e.g., Python/observablehq notebooks.
3. **LanguagePrompt**: Natural language prompts used to control or refine model outputs.
4. **Recommendation**: System recommendations that the user accepts or rejects.
5. **Others**

4 Action Related Codes

4.1 **ACT_EVD**: Action Evidence

The type of action reported in the paper.

1. **NoAct**: No action.
2. **EnvisionedAct**: No solid action reported, but envisioned actions.
3. **Act w/o VA**: Action reported, but not supported by the VA system.
4. **Act w. VA**: Action reported, and supported by the VA system.
5. **Others**

4.2 **ACT_TYP**: Post-Visualization Action Types

Categorizes the types of actions taken after interacting with the visualization. These actions may be supported or directly executed within the visualization or visual analytics system, or performed externally through other mechanisms (e.g., offline coding or modifications to data or models).
*Prefix meaning: **PnD** = Problem Framing and Data Curation, **FnR** = Feature Engineering and Representation, **MT** = Model Formulation and Tuning, **KE** = Prompt and Knowledge Engineering (LLM-based systems).*

1. **PnD_DefineTask**: Formulating or modifying the analytical objective (e.g., switching from multi-class to binary classification based on visual cluster separation).
2. **PnD_EditData**: Adding, removing, or modifying raw data instances (e.g., filtering out outliers, fixing corrupted records).
3. **PnD_EditLabel**: Creating, verifying, or modifying labels (e.g., active learning interfaces).
4. **PnD_SelectSample**: Defining specific cohorts, sub-populations, or sampling strategies for training/testing based on distributions visualised.

5. **FnR_AdjustFeature**: Creating new features, removing noisy ones, or applying transformations after observing feature distributions.
6. **FnR_DefineRelationship**: Define distance metrics, similarity measures, or grouping rules.
7. **MT_SelectAlgorithm**: Changing the underlying model family (e.g., switching from a linear model to a tree-based model after seeing non-linear data patterns).
8. **MT_AdjustHyperpara.**: Tuning hyperparameters (e.g., changing learning rate, tree depth, or k in k-NN).
9. **MT_ModifyArch**: Altering neural network architecture (e.g., adding layers or changing activation functions).
10. **MT_ModifyObjective**: Changing the loss function or adding regularization/constraints based on observed model behavior.
11. **KE_EditPrompt**: Refining natural language prompts or context windows based on generated outputs.
12. **KE_SelectKnowledgeBase**: Modifying the external knowledge bases, documents, or the retrieval sources (RAG) the model uses.
13. **Others**

4.3 **ACT_KI**: Knowledge Injection

*Categorizes how human knowledge is externally injected into the machine learning process through actions that are **NOT** supported or directly executed by the visualization or visual analytics system.*

1. **LabelAnnotation:** Human knowledge represented as labels, tags, classes, categories, ratings, or other annotation values.
2. **ParameterSetting:** Human knowledge represented as explicitly structured machine-readable inputs, such as parameter values, thresholds, weights, partitions, rankings, or categorical settings.
3. **RuleConstraint:** Human knowledge represented as rules, logical conditions, constraints, priors, guardrails, or objectives.
4. **ExampleDemonstration:** Human knowledge represented as examples, counterexamples, demonstrations, or few-shot instances.
5. **TextInstruction:** Human knowledge represented as free-text instructions, prompt content, critiques, rationales, or natural-language requests interpreted by the system.
6. **CodeSpecification:** Human knowledge represented as executable code, formulas, scripts, notebook cells, complex architectural changes, or programmatic queries.
7. **PreferenceJudgment:** Human knowledge represented as approvals, rejections, pairwise comparisons, preferences, votes, or comparative evaluations over alternatives or outputs.
8. **DataInput:** Human knowledge represented as data or a subset of changes and input.
9. **Others**

References

1. What is Self-Supervised Learning (SSL)? A Complete Guide. https://www.snowflake.com/en/fundamentals/self-supervised-learning/
2. …, “Self-Supervised Learning: Generative or Contrastive”, IEEE TRANSACTIONS ON KNOWLEDGE AND DATA ENGINEERING
3. …, “Semi-Supervised and Unsupervised Deep Visual Learning: A Survey”, IEEE TRANSACTIONS ON PATTERN ANALYSIS AND MACHINE INTELLIGENCE
4. Domingos, Pedro, “A Few Useful Things to Know about Machine Learning”, Communication ACM
5. Cortes, Corinna and Vapnik, Vladimir, “Support-Vector Networks”, Machine Learning
6. LeCun, Yann, Bengio, Yoshua, and Hinton, Geoffrey, “Deep Learning”, Nature
7. Koller, Daphne and Friedman, Nir, “Probabilistic Graphical Models”, MIT Press
8. Baydin, Atılım Gunes and Pearlmutter, Barak A and Radul, Alexey Andreyevich and Siskind, Jeffrey Mark, “Automatic Differentiation in Machine Learning: A Survey”, Journal of Machine Learning Research
9. Akbar Telikani, Amirhessam Tahmassebi, Wolfgang Banzhaf, and Amir H. Gandom, “Evolutionary Machine Learning: A Survey”, ACM Computing Surveys
10. Von Rueden, Laura…, “Informed Machine Learning: A Taxonomy and Survey of Integrating Prior Knowledge into Learning Systems”, IEEE TRANSACTIONS ON KNOWLEDGE AND DATA ENGINEERING
11. Hochreiter, Sepp and Schmidhuber, Jürgen, “Long Short-Term Memory”, Neural Computation
12. Hastie, T., Tibshirani, R., & Friedman, J, “The Elements of Statistical Learning”, Springer
13. Mundhenk, T. N. …. “Symbolic Regression via Neural-Guided Genetic Programming Population Seeding.” NeurIPS.
14. D. Sacha, M. Kraus, D. A. Keim and M. Chen, "VIS4ML: An Ontology for Visual Analytics Assisted Machine Learning," in IEEE Transactions on Visualization and Computer Graphics, vol. 25, no. 1, pp. 385-395, Jan. 2019, doi: 10.1109/TVCG.2018.2864838.
15. Financial Times Graphics team, "Financial Times Visual Vocabulary: Tableau Edition," in Financial Times, Apr. 2022, url: www.vizwiz.com.
16. M. Chen, K. Mueller, and A. Ynnerman. Fusion of visual channels. In Hansen, Chen, Johnson, Kaufman, Hagen (eds), Scientific Visualization, Springer, pp.119-127, 2014.
17. Quijano-Chavez, Carlos, et al. "Brushing and Linking for Situated Analytics." *2024 IEEE Conference on Virtual Reality and 3D User Interfaces Abstracts and Workshops (VRW)*. IEEE, 2024.
18. Guerra-Gomez, John A. "Towards Reusable and Reactive Widgets for Information Visualization Research and Dissemination." *2024 IEEE Visualization and Visual Analytics (VIS)*. IEEE, 2024.
19. Snyder, Luke S., and Jeffrey Heer. "Divi: Dynamically interactive visualization." *IEEE Transactions on Visualization and Computer Graphics* 30.1 (2023): 403-413.